

51 单片机 核心 11 问

基于 STC89C52 / HC6800-MS 开发板 | 2026-07-09

1. CPU、MCU、MPU、GPU、NPU、FPU、SoC

CPU (Central Processing Unit)

纯处理器核心。本身不帶存储和外設，需要外接 RAM、ROM、硬盘才能工作。典型场景：PC 的 Intel i7、AMD 服务器芯片。速度快、算力强，但自己什么都‘不帶’，得靠主板上的内存条、固态硬盘来配合。

MCU (Micro Controller Unit)

把 CPU + RAM + ROM + GPIO + Timer + UART + ADC 等全部塞进一颗芯片。相当于‘一台电脑缩到一块 IC 上’。典型单片机、STM32。集成度高、引脚少、便宜、省电，专门做简单控制——控制个 LED、读个温度传感器、驱动个小电机。

MPU (Micro Processing Unit)

只有 CPU 核心的芯片，存储和外設得自己在外搭。ARM Cortex-A 系列跑 Linux 的就是 MPU，板上得外接 DDR 内存、闪存。性能强能跑 OS，但外围电路复杂。

GPU (Graphics Processing Unit)

图形处理器。内部 = 显存 + GPU 核心 + 散热 + 输出接口。专做图形渲染和图像处理，能同时并行海量浮点运算。NVIDIA GeForce 系列就是 GPU。打游戏、剪视频、跑 AI 训练的核心算力都靠它。

NPU (Neural Processing Unit)

神经网络处理器，专为 AI 推理和卷积运算做了硬件级优化。和 GPU 不同，NPU 在矩阵乘法和激活函数上功耗极低效率极高。手机 Hexagon NPU、华为达芬奇架构 NPU 干的就是这个。

FPU (Floating Point Unit)

浮点运算单元，通常嵌在 CPU 内部，专门加速小数运算。没有 FPU 的话 CPU 只能算整数，浮点得用软件模拟慢很多。ARM Cortex-M0/M3 不帶，做浮点运算就费劲得多。

SoC (System On Chip)

片上系统——把多个不同功能的芯片集成到一颗上。比如手机里的骁龙 8 里面有 CPU+GPU+NPU+ISP+基带+内存控制器。关键不是‘集成得多’，而是集成的都是独立功能模块，每个拿出来都能独立干活。SoC ≠ MCU。规模大得多、能跑完整 OS，MCU 资源少、跑裸机或轻量 RTOS。

MCU = CPU + RAM + ROM + GPIO + 外设 → 一台微型电脑塞进一颗芯片

MPU = 只有 CPU → 外接 DDR + Flash + 外设控制器

SoC = CPU + GPU + NPU + DSP + Modem + ... → 手机级全家桶

2. 单片机最小系统

单片机最小系统 = 能让芯片正常工作的最简电路，缺一不可的三件套：

(1) 电源电路

VCC (5V) 和 GND 给芯片供电。STC89C52 工作电压一般是 5V DC。每个 VCC 脚旁边需要并联一个的去耦电容滤除高频噪声。供不上电一切免谈。

(2) 晶振电路（时钟）

晶振 + 两个匹配电容 (20~30pF)，接在 XTAL1 和 XTAL2 脚上。晶振 = 芯片的‘心跳’，给 CPU 提供时钟节拍。51 常用 12MHz 11.0592MHz（算波特率方便，9600bps 能整除）。没晶振芯片就是一块砖——时钟停摆任何指令都无法执行。

(3) 复位电路

上电时给 RST 引脚一个短时间高电平 (>2个机器周期)，把芯片内部所有寄存器拉回初始值，然后恢复低电平让程序从地址 0000 开始执行。通常用 RC 电路 (10uF 电容 + 10K 电阻) 实现上电自动复位，再并联一个按键做手动复位。没复位芯片启动时内部状态不可控，可能飞到随机地址跑飞。

最小系统 = 5V 电源 + 12MHz 晶振(含两个电容) + RC 复位电路(含按键)

这三个搭好芯片就能跑代码了。LED、数码管、按键等外设都是加在最小系统上的。

3. STC89C52 芯片内部结构

STC89C52 是宏晶科技基于 Intel 8051 内核的增强型 8 位单片机。内部主要包含：

CPU 核心

8 位 8051 处理器内核：ALU 运算器、累加器 ACC、寄存器 B、程序状态字 PSW、堆栈指针 SP、数据指针 DPTR(16位)、程序计数器 PC(16位)。所有运算和逻辑控制都在这里完成。

存储器

8KB Flash ROM —— 放程序代码，ISP 在线编程烧写，可反复擦写。

512 字节内部 RAM —— 低128字节给用户用，高128字节是 SFR 区。

4KB EEPROM —— 掉电不丢，存配置参数。

I/O 端口

P0、P1、P2、P3 —— 4 组 8 位双向 I/O，共 32 个 GPIO 脚。P0 做 I/O 需外加上拉电阻。P3 口有复用功能：P3.0(RXD)、P3.1(TXD) 串口，P3.2(INT0)、P3.3(INT1) 外部中断。

定时器 / 计数器

3 个 16 位定时器 T0、T1、T2。用于定时延时、计数外部脉冲、产生串口波特率。

串口通信

1 个全双工 UART 串行口，通过 P3.0(RXD) 和 P3.1(TXD) 收发数据。

中断系统

6 个中断源：2 个外部中断 INT0/INT1、3 个定时器中断 T0/T1/T2、1 个串口中断。支持两级优先级。

其他

看门狗定时器 —— 程序跑飞自动复位。时钟振荡器接外部晶振。复位逻辑。ISP 在线编程支持。

4. 内存和外存的特点

特性	内存 (RAM)
速度	快，CPU 按地址直接访问
掉电	数据全丢 (易失性)
价格	贵 (按每字节算)
用途	存放运行时变量、堆栈、临时数据
状态	随时可读写

特性	外存 (ROM / Flash / HDD)
速度	慢，比 RAM 低几个数量级
掉电	数据保留 (非易失性)
价格	便宜大碗 (几十块几百 GB)
用途	存放程序代码和文件
状态	运行前加载到 RAM

程序运行流程

a.out (外存) --> 加载到 RAM --> CPU 从 RAM 取指令和数据 --> 执行 --> 结果写回 RAM

存储层次 (从上到下：越往下越慢、越大、越便宜)

寄存器 (CPU 内部) -- 最快，几 KB
Cache 缓存 -- 很快，几十KB~MB
内存 RAM -- 快，几百MB~几GB
外存 ROM/Flash -- 慢，几十GB~TB

5. RAM 和 ROM 的特性

对比项	RAM（随机存储器）	ROM（只读存储器）	
全称	Random Access Memory	Read Only Memory	
可写性	随时可读可写	运行中只读，写入需烧写	
掉电	数据全丢（易失性）	数据保留（非易失性）	
速度	快	比 RAM 慢	
用途	存变量、堆栈、临时结果	存程序代码、固定数据	
51 中	512B 片内 SRAM	8KB Flash ROM	

举例：int a = 2; int b = 3; int sum = a + b; —— a、b、sum 的值存在 RAM 里，'int a = 2;' 对应的机器指令存在 ROM 里
ROM 取指令 → 从 RAM 取 a、b → ALU 算加法 → sum 写回 RAM。

SRAM vs DRAM

SRAM（静态 RAM）：用触发器存数据，快、贵、不用刷新，做 Cache 和片内 RAM。

DRAM（动态 RAM）：用电容存数据，慢、便宜、需定时刷新，做电脑内存条。51 片内 RAM 是 SRAM。

6. CPU 与 RAM 之间三大总线

总线	方向	作用	51 宽度
地址总线	CPU→RAM（单向）	CPU 发出要访问的地址	16位，寻址 64KB
数据总线	CPURAM（双向）	实际搬运数据的通道	8位，一次 1 字节
控制总线	CPURAM（双向）	发读写信号、回传状态	多条信号线

读操作流程

1. CPU 把地址放到地址总线上 → 比如 0x0030
2. CPU 在控制总线上发出 读 信号（RD=0）
3. RAM 根据地址找到对应存储单元
4. RAM 把该单元的数据放到数据总线上
5. CPU 从数据总线读走数据，读信号恢复

写操作流程

1. CPU 把目标地址放到地址总线上 → 比如 0x0040
2. CPU 把要写的数据放到数据总线上
3. CPU 在控制总线上发出 写 信号（WR=0）
4. RAM 把数据写入对应地址
5. 写信号恢复

7. 如何实现数码管的显示

以 HC6800-MS 开发板上的 4 位共阴极数码管为例。共阴极 = 所有 LED 的负极连在一起接 GND，要点亮某一段只需把对应引脚置高

硬件连接

P0 口 — 段选（a~g + dp 八个段）

P2 口低 4 位 — 位选（通过 NPN 三极管驱动，高电, 选通）

段选 —— 决定显示什么数字

8 段数码管，每段对应一个 LED：

a
f b
 g
e c
 d dp

数字 '0' → 点亮 a b c d e f → 0011 1111 = 0x3F

数字 '1' → 点亮 b c → 0000 0110 = 0x06

数字 '2' → 点亮 a b d e g → 0101 1011 = 0x5B

位选 —— 决定哪一位亮

同一时刻只选通一位。P2. 0→第1位、P2. 1→第2位、P2. 2→第3位、P2. 3→第4位。

显示一个数字的步骤：关所有位选（消隐）→ P0=字形码 → 选通目标位 → delay → 关位选 → 处理下一位。

8. 数码管动态显示原理

动态显示的核心是'视觉暂留效应'——人眼对光的反应有约 50ms 余辉，只要刷新够快，看起来就是同时亮的。

为什么不能静态显示四位？

四位数码管的段选线是并联的——如果四位同时亮，段选信号相同，四位会显示一模一样的数字。所以必须分时点亮，每位亮左右然后快速切换。

动态扫描循环

```
while(1) {
    关位选(消隐) → P0=千位字形码 → 开第1位 → delay(1ms) → 关
    关位选(消隐) → P0=百位字形码 → 开第2位 → delay(1ms) → 关
    关位选(消隐) → P0=十位字形码 → 开第3位 → delay(1ms) → 关
    关位选(消隐) → P0=个位字形码 → 开第4位 → delay(1ms) → 关
}
```

为什么看不出来闪烁？

四位循环一次约 4ms，刷新频率约 250Hz，远高于人眼的闪烁感知阈值 (~50Hz)。每个位置 1ms，但刷新够快，大脑就觉得四位一直同时亮着。

消隐的重要性

切换位选前必须先关掉所有位，再换段选数据。否则前一位的段码会短暂出现在下一位上产生'鬼影'——消隐一步不能省。

9. 指定位清 0 和指定位置 1 如何实现

51 单片机是 8 位机，操作单位是 unsigned char (1 字节 = 8 位, bit7~bit0)。设初值 t = 0x00 (0000 0000)。

指定位清 0: t &= ~(1 << n)

原理: 1<<n 造出只有第 n 位是 1 的数，~ 取反把它变成只剩第 n 位是 0 其余全 1，&= 与运算：与 0 为 0，与 1 不变。

```
t = 0xFF;           // 1111 1111
t &= ~(1 << 1);     // bit1 清0 → 1111 1101 (0xFD)
t &= ~(1 << 3);     // bit3 清0 → 1111 0101 (0xF5)
```

指定位置 1: t |= (1 << n)

原理: 1<<n 只有第 n 位是 1，|= 或运算：或 1 得 1，或 0 不变。

```
t = 0x00;           // 0000 0000
t |= (1 << 0);       // bit0 置1 → 0000 0001 (0x01)
t |= (1 << 7);       // bit7 置1 → 1000 0001 (0x81)
t |= (1<<2) | (1<<3); // bit2,bit3 同时置1 → 1000 1101 (0x8D)
```

操作	写法
第 n 位置 1	t = (1 << n) 口诀：左移 n 位然后或
第 n 位清 0	t &= ~(1 << n) 口诀：左移 n 位取反然后与
翻转第 n 位	t ^= (1 << n) 口诀：左移 n 位然后异或
读第 n 位状态	(t >> n) & 1 口诀：右移 n 位看最低位

10. 什么是外设寄存器

一句话：外设寄存器 = 一段映射到固定地址的内存空间。你往这个地址写数据，就等于在给外设硬件发指令。

芯片厂商在硬件设计时，把 GPIO、串口、定时器等外设的控制信号通过地址线接到一段固定地址上。对来说，写一个外设寄存器和写普通内存地址的操作完全一样——都是“把地址放地址总线、把数据放数据总线、发写信号”。区别只到 RAM 地址是存数据，写到外设寄存器地址是控制硬件。

```
#define P2 *((unsigned char *)0xA0)

P2 = 0xFF; // 往地址 0xA0 写 1111 1111 → P2 口全部输出高电平
P2 = 0x00; // 往地址 0xA0 写 0000 0000 → P2 口全部输出低电平 → LED 全亮
```

统一编址 vs 独立编址

51 单片机用的是统一编址（内存映射 I/O）：外设寄存器和 RAM 共用同一套地址空间。地址 0x00~0x7F 是普通 RAM，0x80~0xFF（特殊功能寄存器 / 外设寄存器区）。CPU 访问 RAM 和访问外设寄存器汇编指令完全 MOV。各家芯片的寄存器布局不同，具体要查芯片数据手册（datasheet）。

11. LED 和数码管分别用到了哪几个寄存器

以普中 HC6800-MS 开发板（STC89C52）为例：

控制 LED

LED 灯接了 P2 口 8 个引脚，低电平点亮（共阳极接法，阳极接 VCC，阴极通过限流电阻接 P2 口，P2 口输出低电平导通发光）。只用到 1 个寄存器：

```
P2 寄存器（地址 0xA0）
P2 = 0xFF → 8 位全 1、全高电平 → 灯全灭
P2 = 0x00 → 8 位全 0、全低电平 → 灯全亮
P2 = ~(1<<0) → 只有 bit0=0 → 只有第 1 个 LED 亮
```

控制数码管

用到了 2 个寄存器：

```
P0 寄存器（地址 0x80） → 段选（a~g + dp）
    共阴极高电平点亮
    显示 '0' → P0 = 0x3F（0011 1111 = a~f 亮、g 灭）

P2 寄存器（地址 0xA0） → 位选（用低 4 位）
    P2.0→第1位 P2.1→第2位 P2.2→第3位 P2.3→第4位
    高电平通过 NPN 三极管选通对应位
    P2 = 0x01 / 0x02 / 0x04 / 0x08 分别选第1~4位
```

注意：P2 口被 LED 和数码管位选复用了。所以 LED 和数码管通常不能同时用
数码管动态扫描时 P2 在不断切换位选，LED 也会跟着闪烁
数码管动态显示 = P0(段选字形码) + P2 低4位(位选切换) + 定时刷新循环